Stacks and Queues

[CSE2010] Data Structures

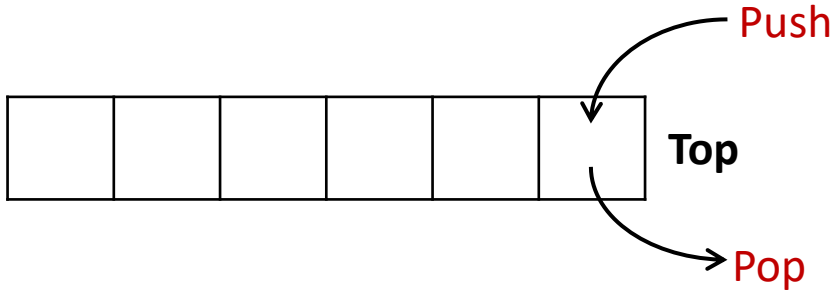
Department of Data Science

Stacks

Stack is an ordered list in which insertion and deletion are done at one end, called *top*.

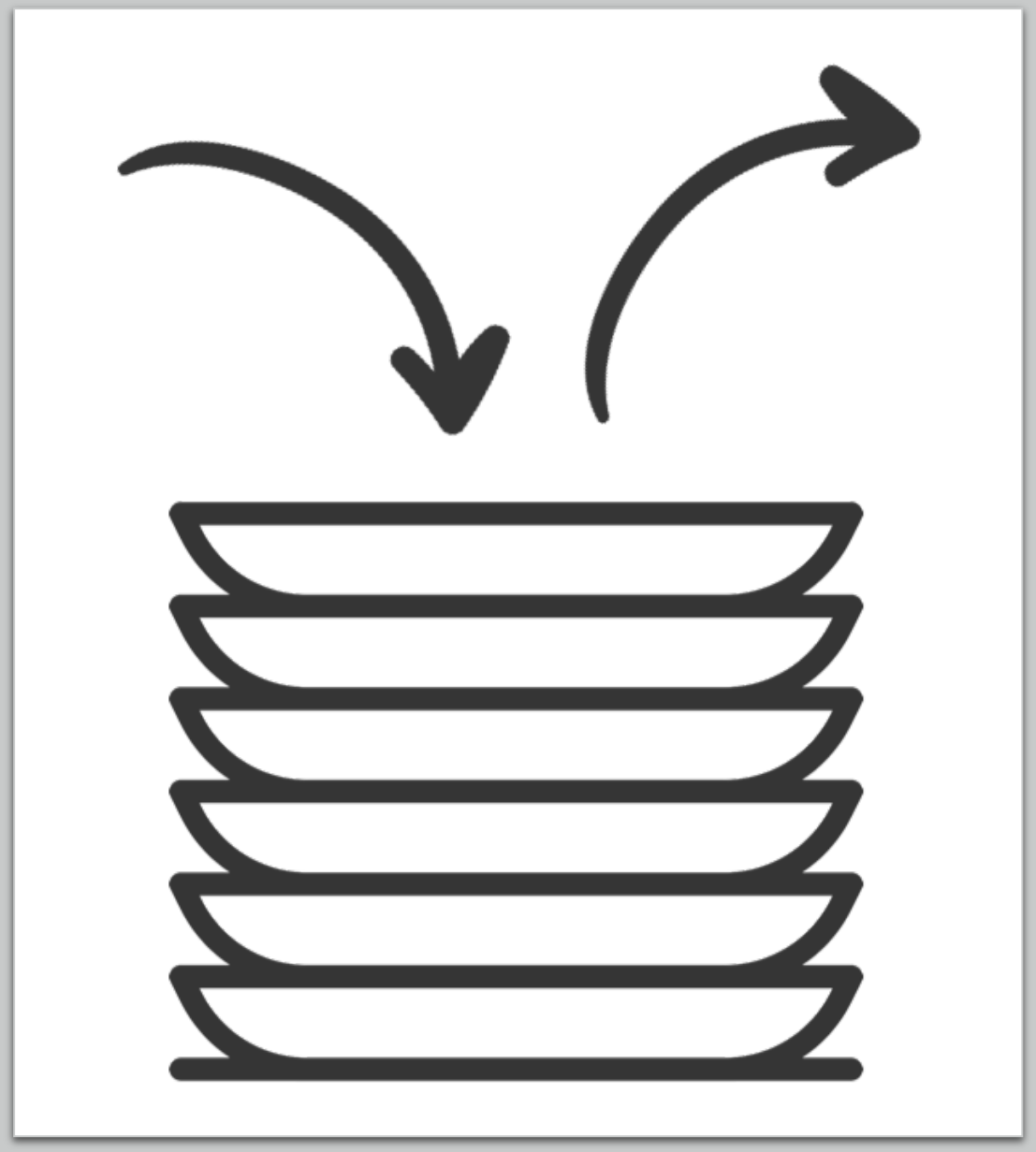
Stacks

- Stack is a list of elements with three constraints:
  1. **Push (insert)**: insert element only to the end of a stack.
  2. **Pop (delete)**: delete element only from the end of a stack.
  3. **Top**: read the last element of a stack.
- The only access to stack is the most-recently-added item.
- The last element inserted is the first one to be deleted (Last in First out, LIFO)



Pile of Plates

- Plates are added to the stacks as they are cleaned, and they are placed on the top.
- When a plate is required, it is taken from the top of the stack.
- The first plate placed on the stack is the last one to be used.



Array Implementation of Stack

- Array $S[1..n]$ of n elements
  - $S.top$ indexes the most recently inserted element.
  - Stack consists of elements $S[1], S[2], \ldots, S[S.top]$
- When $S.top = 0$, stack is empty.
  - If we attempt to pop from an empty stack, the stack **underflows**.
- When $S.top = n$, stack is full.
  - If we attempt to push to a full stack, the stack **overflows**.

	1	2	3	4	5	6
S						

$S.top = 0$

Pseudocode of Stack Operations

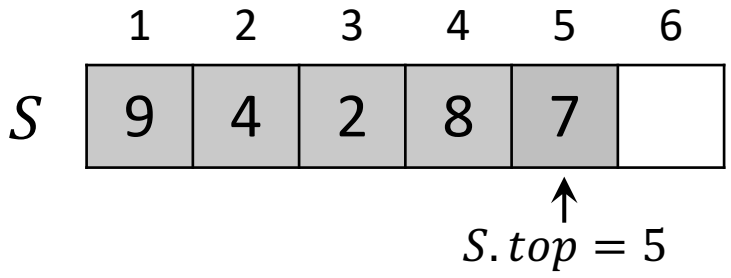
```
IsEmpty(S)
    return (S.top == 0)

IsFull(S)
    return (S.top == S.length)

Push(S, x)
    if isFull(S) then error "overflow"
    else
        S.top ← S.top + 1
        S[S.top] ← x
```

```
Pop(S)
    if IsEmpty(S) then error "underflow"
    else
        S.top ← S.top – 1
        return S[S.top+1]
```

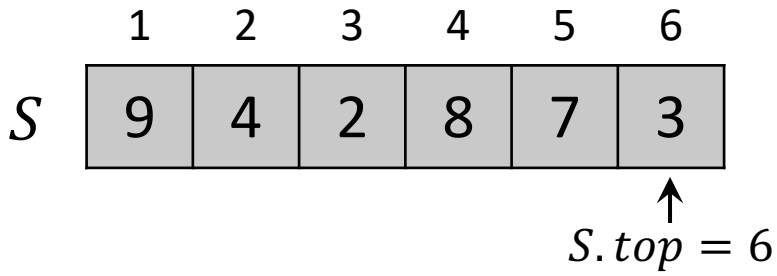
- Each operation takes O(1) time.

Array Implementation of Stack

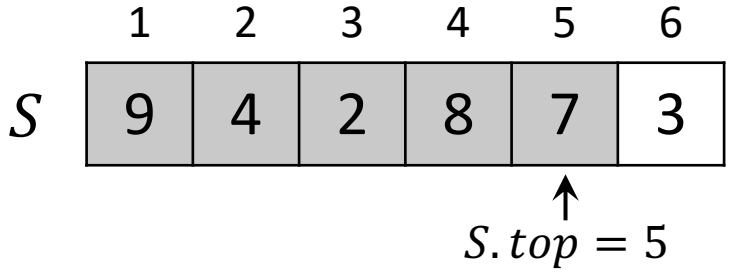
1. Stack S has 5 elements.



2. Push(S, 3).

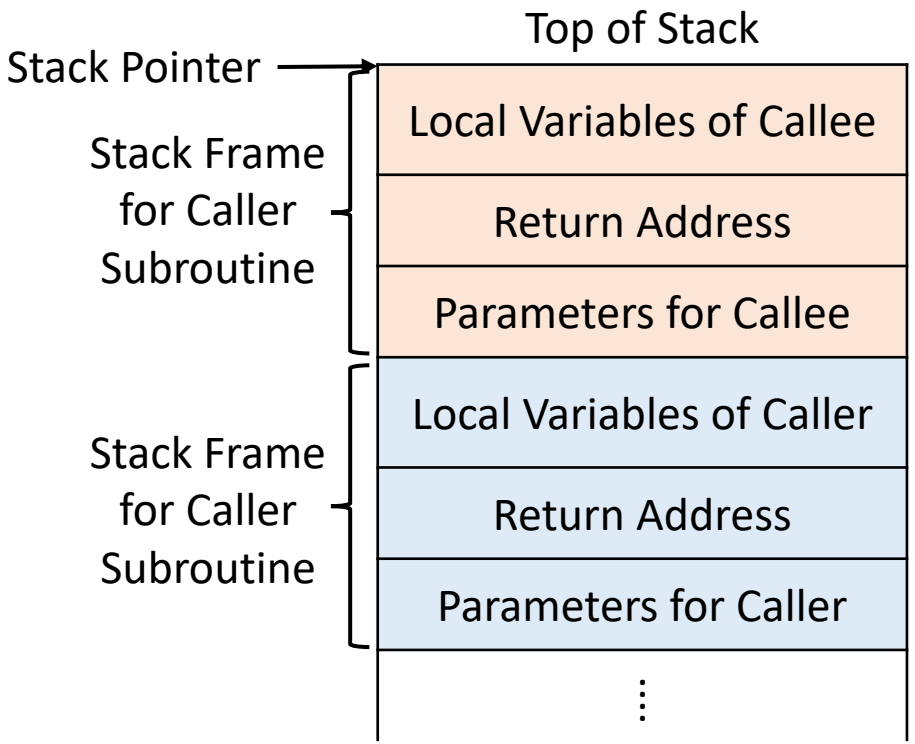


3. Pop(S) returns 3
   - 3 is no longer in the stack, though it still appears in the array.
   - The top element is 7.



When do we use stacks?

- Implementing function calls
  - Stack frames in a call stack
- "Undo" function in a text editor
- Page-visited history in a web browser
  - Back buttons
- Matching delimiters in arithmetic expression
- Matching tags in HTML and XML



Example: Checking Balance of Braces

Input string	Stack as algorithm executes				
	1.	2.	3.	4.	
`{a{b}c}`	`{`	`{` `{`	`{`		1. push "`{`" 2. push "`{`" 3. pop 4. pop Stack empty $\Longrightarrow$ balanced
`{a{bc}`	`{`	`{` `{`	`{`		1. push "`{`" 2. push "`{`" 3. pop Stack not empty $\Longrightarrow$ not balanced
`{ab}c}`	`{`				1. push "`{`" 2. pop Stack empty when last "`}`" encountered $\Longrightarrow$ not balanced

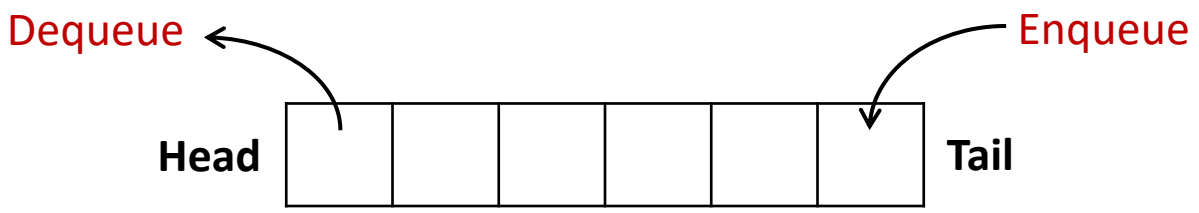
Carrano and Prichard, Data Abstraction and Problem Solving with JAVA: Walls and Mirrors

Queues

Queue is an ordered list in which insertions are done at on end (tail) and deletions are done at other end (head).

Queues

- Queue is a list of elements with the three restrictions:
  1. **Enqueue (insert)**: insert element only to the tail of a queue.
  2. **Dequeue (delete)**: delete element only from the head of a queue.
  3. **Front**: read the element at the head of a queue.
- The element inserted the first is the first one to be deleted (First in First out, FIFO).

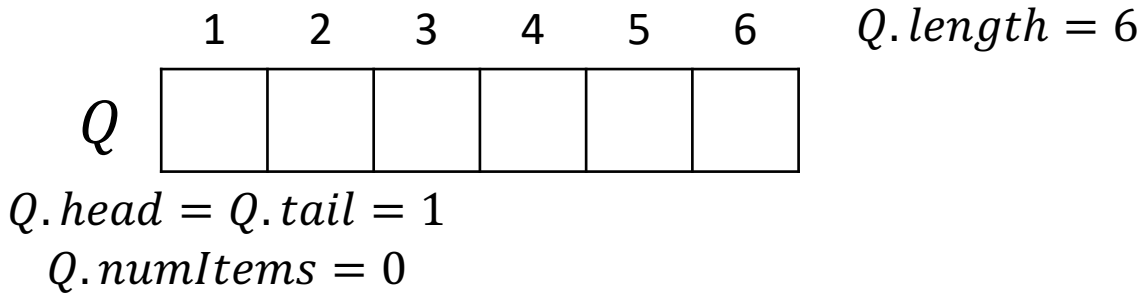


Waiting Line

- People line up to be served.
- The first person placed in the line is the first one to be served.

Array Implementation of Queue

- Array $Q[1..n]$ of n elements
  - $Q.head$ indexes the element inserted the first.
  - $Q.tail$ indexes the next location at which a newly arriving element will be inserted.
  - Queue consists of elements $Q[Q.head], \ldots, Q[Q.tail - 1]$
- If we attempt to delete an element from an empty queue
  - the queue **underflows**.
- If we attempt to insert an element to a full queue
  - the queue **overflows**.



Pseudocode of Queue Operations

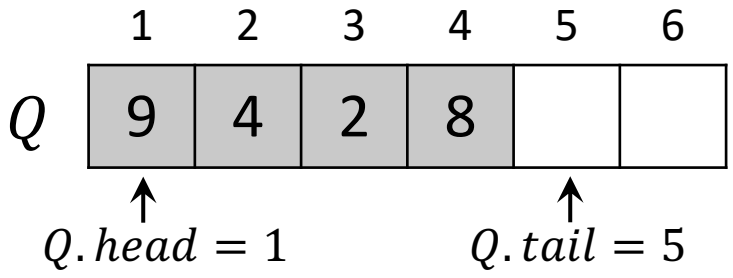
```
IsEmpty(Q)
    return (Q.numItems == 0)

IsFull(Q)
    return (Q.numItems == Q.length)

Enqueue(Q,x)
    if isFull(Q) then error "overflow"
    else
        Q[Q.tail] ← x
        Q.numItems ← Q.numItems + 1
        if Q.tail == Q.length
            Q.tail ← 1
        else Q.tail ← Q.tail + 1
```

```
Dequeue(Q)
    if isEmpty(Q) then error "underflow"
    else
        x ← Q[Q.head]
        Q.numItems ← Q.numIterms - 1
        if Q.head == Q.length
            Q.head ← 1
        else Q.head ← Q.head + 1
        return x
```

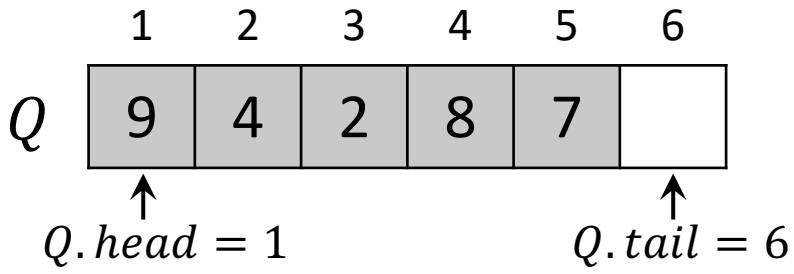
- Each operation takes O(1) time.

Array Implementation of Queue

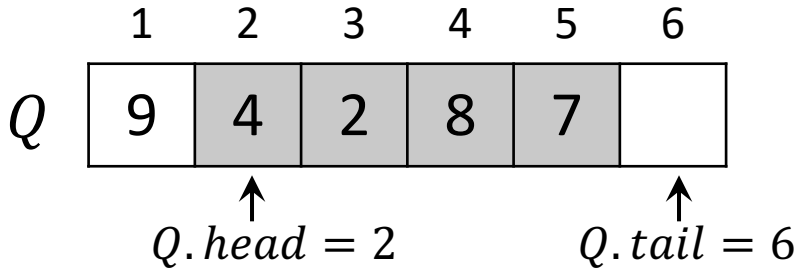
1. Queue Q has 4 elements.



2. Enqueue(Q, 7)

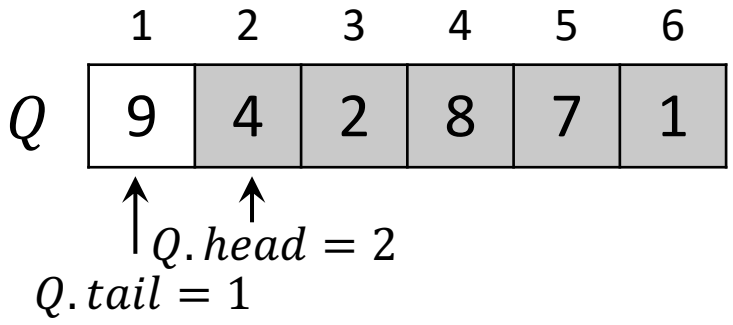


3. Dequeue(Q) returns 9
   - 9 is no longer in the queue, though it still appears in the array.
   - The front element is now 4.

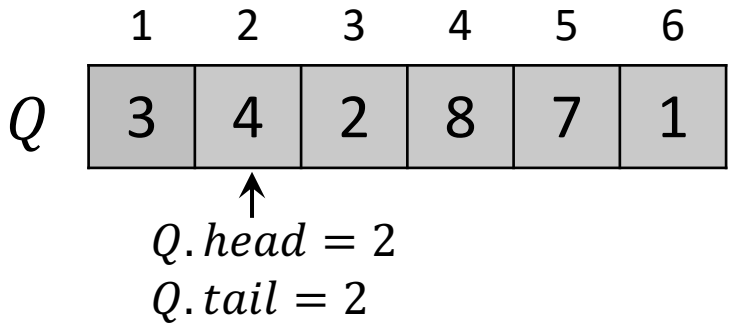


Array Implementation of Queue

4. Enqueue(Q, 1)



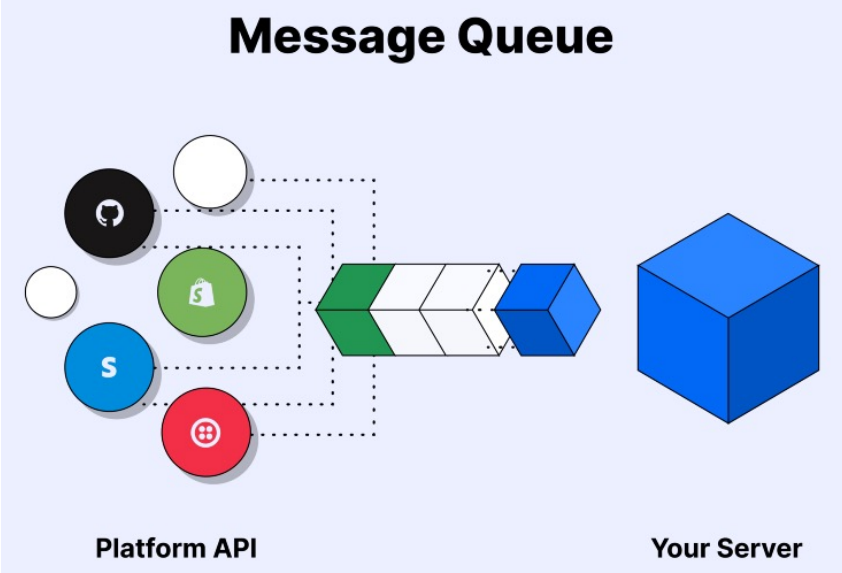
5. Enqueue(Q, 3)



6. Enqueue(Q, 10)
   - Full! Raise an error “overflow”.

When do we use queues?

- Printing documents
- Asynchronous requests
  - Message queues
- Airplanes waiting for takeoff
- Patients waiting for their doctor



https://medium.com/hookdeck/an-introduction-to-asynchronous-processing-and-message-queues-218af596bf1b

Abstract Data Type (ADT)

Stack and queue are examples of ADT.

Abstract Data Types (ADT)

- Most programming languages don’t come with the stack or queue as a built-in data type or class.
- Stack and queue don’t care about what data structure is under the hood.
  - Whether we accomplish this with an array or other type of built-in data structure doesn’t matter.

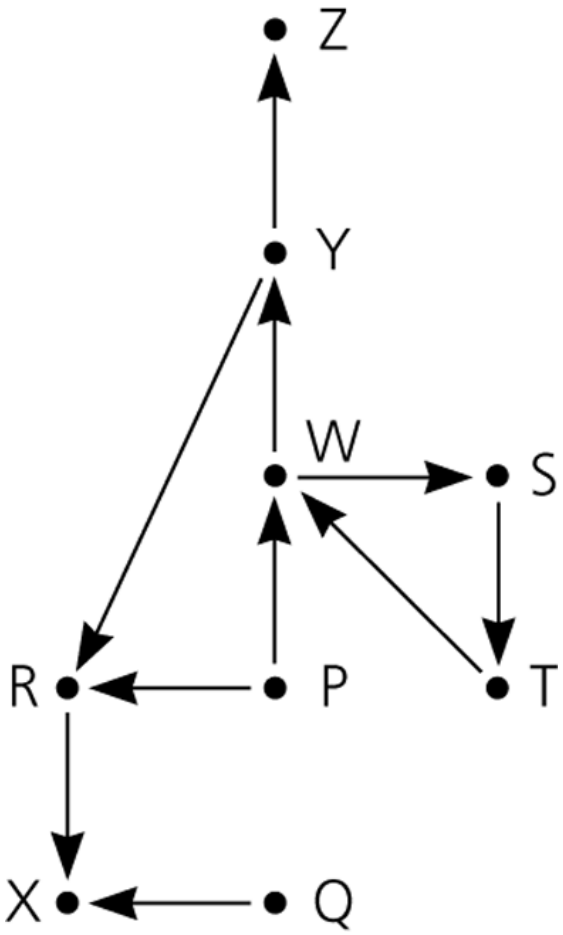
Example: Recognizing Palindromes

- $Palindromes = \{\boldsymbol{w} \mid \boldsymbol{w}\ reads\ the\ same\ left\ to\ right\ as\ right\ to\ left\}$
  - mom
  - level
  - tenet
  - Madam, I'm Adam

Example: Recognizing Palindromes

```
def isPalindrome(w: str) -> bool:
    for ch in w:
        Q.enqueue(ch)
        S.push(ch)
    # Begin to compare
    while not Q.isEmpty():
        if Q.dequeue() != S.pop():
            return False;
    # Done with empty queue & empty stack
    return True
```

Example: Searching Flight Map



- ↑: direct flight
- Is there a path from city P to city Z?

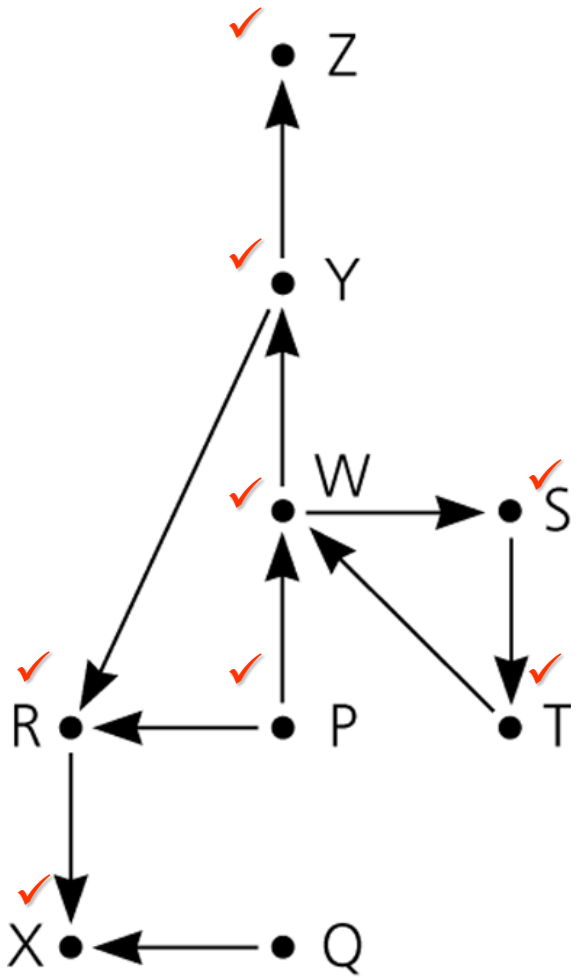
Carrano and Prichard, Data Abstraction and Problem Solving with JAVA: Walls and Mirrors

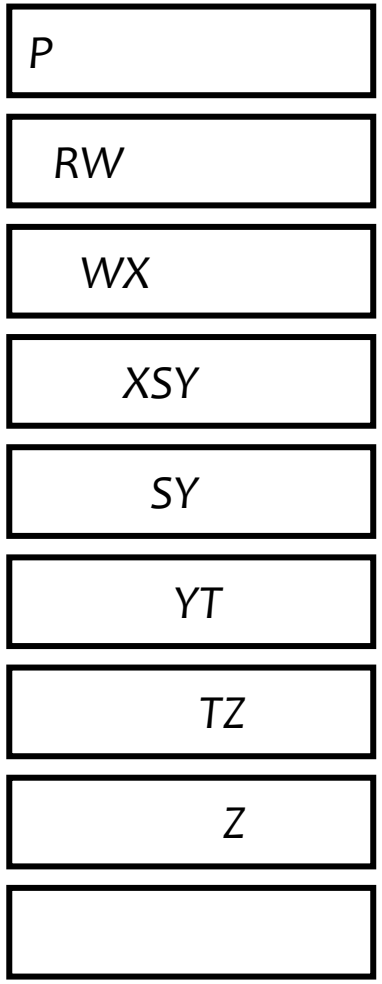
Example: Searching Flight Map

- Select a queue to keep unvisited cities
- Can we use a **stack** to find a path from origin to destination?

```
havePath(origin, destination)
    Build an empty queue Q
    Mark all cities UNVISITED
    Q.enqueue(origin)
    Mark origin as VISITED
    while Q is not empty
        x = Q.dequeue()
        for each city y that is directly reachable from x
            if y != destination
                Q.enqueue(y)
                Mark y as VISITED
            else
                return true // Path exists
    return false // There is no path
```

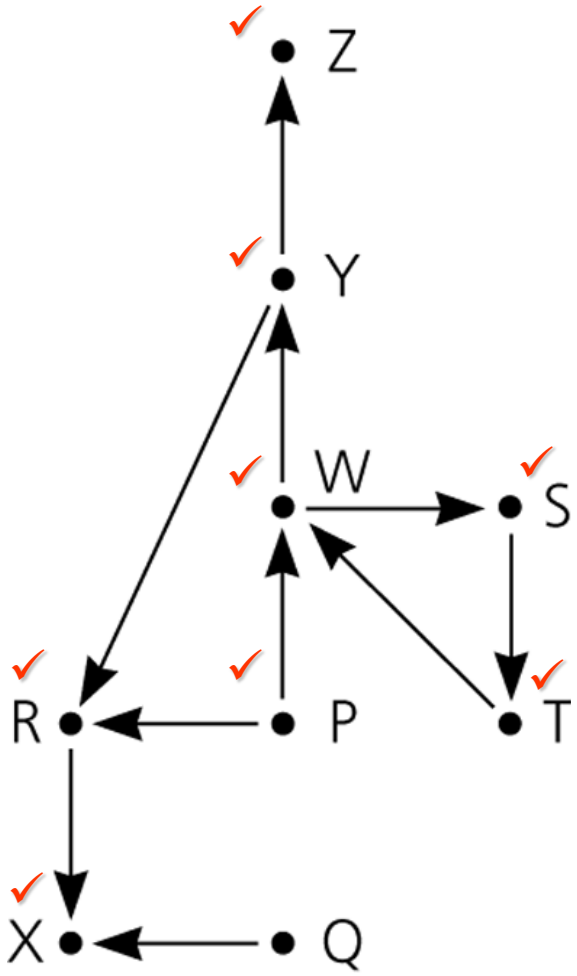
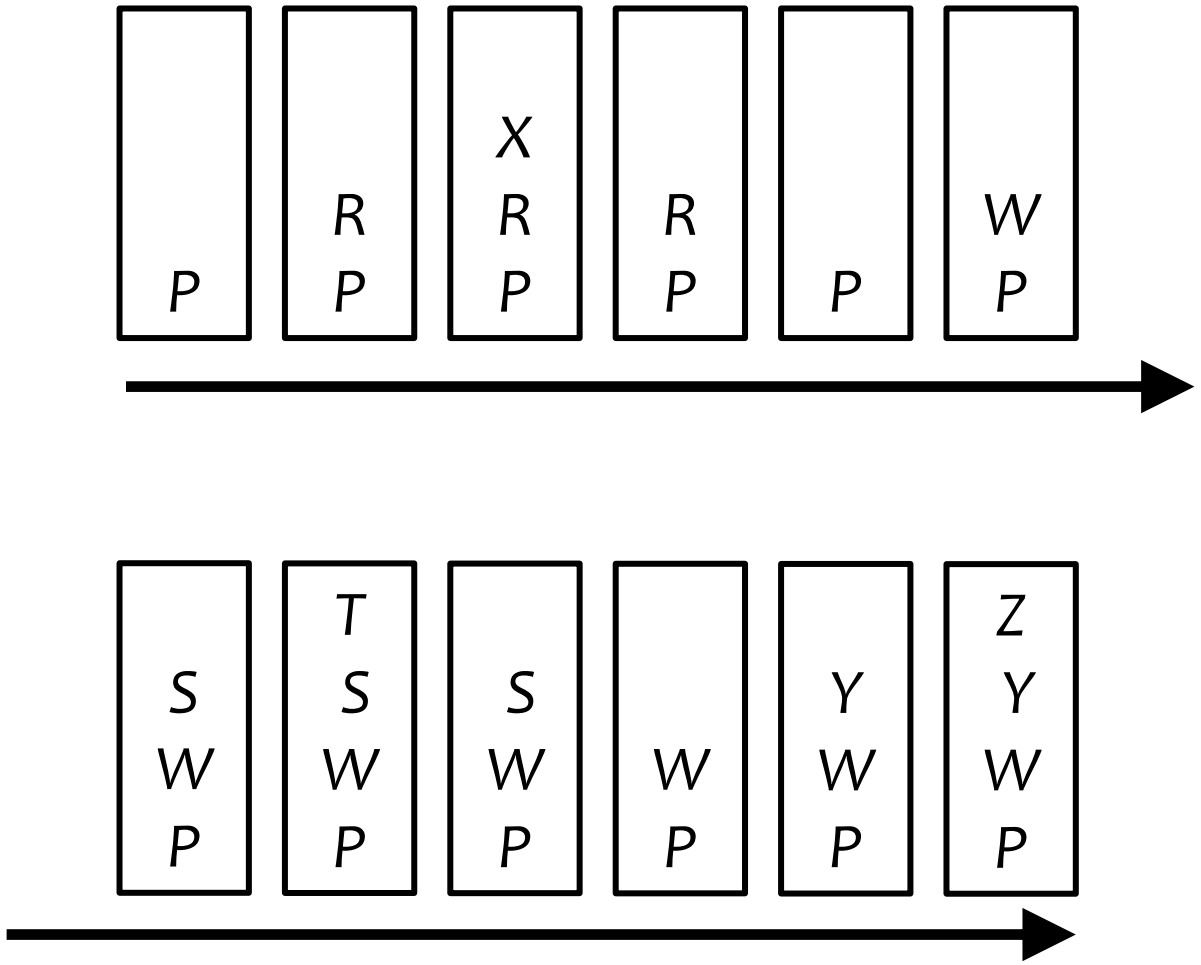
Example: Searching Flight Map





Searching Flight Map

- Select a stack to keep unvisited cities

```
havePath(origin, destination)
    Build an empty stack S
    Mark all cities UNVISITED
    S.push(origin)
    Mark origin as VISITED
    while S is not empty
        x = S.top()
        If no flight exists from x to unvisited cities
            S.pop()
        else
            Select unvisited city y directly reachable from x
            if y != destination
                S.push(y)
                Mark y as VISITED
            else
                return true // Path exists
    return false // There is no path
```

Example: Searching Flight Map